



Rapport final du projet informatique



Sommaire

Introduction	3
Rappel des spécifications et de la conception détaillée	3
Méthode générale de résolution.....	3
Détail éventuel des parties difficiles ou originales	5
Problèmes rencontrés et solutions apportées	6
Les écarts par rapport au cahier des charges	7
Bilan : Ce que le projet nous a apporté	8
Les +.....	8
Les -	8
Conclusion	9
Discussion des résultats	9
Prolongements possibles	9
Améliorations	10

Introduction

Notre objectif dans ce projet était de développer le jeu *Captain Flip* où les joueurs peuvent créer et gérer leurs comptes, jouer des parties contre des adversaires, et suivre leurs statistiques de victoires et de défaites. Le jeu intègre des fonctionnalités complexes, comme l'affichage du plateau de jeu, la gestion des tuiles et les capacités spécifiques des personnages. Ce rapport détaille les spécifications, la conception, les méthodes de résolution, les défis rencontrés, les écarts par rapport au cahier des charges initial, ainsi que les enseignements tirés de ce projet.

Rappel des spécifications et de la conception détaillée

Les spécifications quant à ce projet étaient la création ainsi que la gestion de comptes joueurs afin de pouvoir jouer au jeu *Captain Flip*, une sauvegarde du nombre de victoires et de défaites de chacun des joueurs possédant un compte et ayant joué, ainsi que la possibilité de consulter ces nombres pour chacun des joueurs. Il était aussi demandé de pouvoir jouer contre un joueur ordinateur et/ou humain.

Quant à la conception détaillée du jeu, il fallait tout d'abord afficher le plateau de jeu, piocher une tuile et la retourner si on le souhaitait, ainsi que de pouvoir poser cette tuile dans n'importe quelle colonne, en prenant en compte qu'il fallait qu'elle soit posée dans la case la plus en bas possible (la case libre la plus basse).

Ensuite, il fallait coder toutes les fonctionnalités propres aux différents personnages, ainsi que de les faire fonctionner soit directement, soit à la fin du jeu, ainsi que le fonctionnement du jeu au tour par tour.

Méthode générale de résolution

Pour ce projet, nous avons décidé de procéder par étapes.

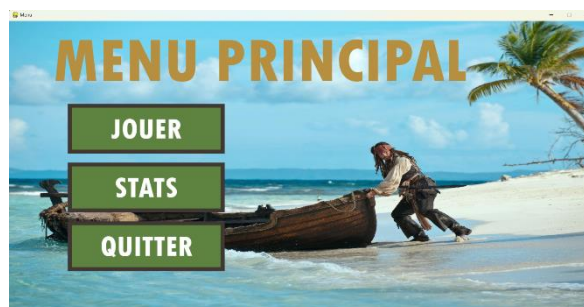
Tout d'abord, nous nous sommes concentrées sur l'apprentissage de l'utilisation de la bibliothèque PyGame, qui était quelque chose de nouveau pour nous. Pour ce faire, nous

avons recherché plusieurs vidéos sur YouTube afin de nous familiariser avec cette bibliothèque.

Ensuite, nous avons scindé ce que nous devons faire en deux grands axes : la partie “comptes” et la partie “jeu” qui ont été faites en parallèle. D’un côté, nous avons porté notre attention sur le jeu, plus précisément sur l’interface graphique. L’affichage du plateau de jeu ainsi que la création des cases ont été nos premières lignes de code. Après ceci, nous avons créé le code pour la pioche, sachant que dans ce jeu nous ne pouvons piocher qu’une seule fois par tour, et nous avons la possibilité de retourner la tuile une seule fois. Ensuite, nous avons codé le début d’un tour de jeu pour rendre les actions de piocher une tuile, pouvoir la flipper, ainsi que de la placer sur la grille, utilisables. Par la suite, nous avons rendu possible le changement de joueur pour permettre au deuxième joueur de jouer, puis nous avons codé le calcul des pièces gagnées à chaque action. Nous avons enfin codé la gestion de la fin de partie avec l’arrêt du jeu, le compte final des pièces ainsi que l’affichage de la victoire d’un des joueurs et des grades obtenus en cette fin de partie.



Ecran de connexion (joueur 1)



Ecran du menu principal

De l’autre côté, nous avons esquissé une première ébauche du menu principal qui nous permettait de nous connecter, de nous inscrire, de consulter les statistiques, de quitter la page, et bien évidemment de lancer le jeu. Après cela, nous avons décidé de commencer à coder la partie “comptes”, en optant pour un fichier pour sauvegarder ceux-ci, ainsi que les victoires et défaites de chaque joueur. Cette partie contient la possibilité de se connecter avec un compte déjà existant et d’inscrire un nouveau compte. En inscrivant un nouveau compte, le nombre de victoires et de défaites ainsi que le nombre de fois que les grades ont été obtenus sont initialisés à 0. Il est aussi possible de changer ces données directement dans le fichier. Des classes Joueur et GestionJoueurs permettent de modifier ces données à la fin de la partie (Victoire/Défaite ainsi que le grade obtenu).

A la fin, nous avons pu fusionner ces deux codes pour faire fonctionner les deux ensembles. C’est-à-dire que le bouton “Jouer” permettait de lancer le jeu sans avoir à démarrer un autre programme, et qu’à la fin d’une partie les comptes de victoires / défaites ainsi que les grades obtenus étaient sauvegardés dans le fichier Joueurs.

Détail éventuel des parties difficiles ou originales



Table de jeu

Au cours de ce projet, nous avons rencontré quelques difficultés, notamment la création de la grille qui permet au joueur de cliquer dans n'importe quelle case pour poser la tuile qu'il a pioché. Pour ce faire, nous avons créé les classes Grille et Case. L'objet Case sert à savoir si

une case est occupée par une tuile ou non, et l'objet Grille permet de définir la position à laquelle la tuile va être placée lors d'un clic sur une case. On change alors l'état de cette case en "occupé" avec l'objet Case pour placer par la suite les tuiles au-dessus de cette case maintenant occupée (exemple : si la case en bas d'une colonne est occupée et qu'on clique tout en haut de la colonne, la tuile se posera juste au-dessus de la case occupée. Si l'on clique sur une case déjà occupée, la tuile ne s'y met pas). Bien évidemment, toutes ces informations doivent être gardées en mémoire afin de faire fonctionner le jeu pendant toute la partie.

Une autre partie difficile fut la création de la pioche. Avec 9 personnages chacun présent 16 fois sur les deux faces des tuiles, 72 tuiles personnages (double-face) sont donc présentes dans ce jeu. Ces personnages sont répartis sur les tuiles de façon à ce que chaque couple (recto/verso) n'existe qu'une seule fois. Il a donc fallu gérer cette répartition spécifique dans notre code. Pour cela, nous avons donc décidé de procéder en deux étapes ; tout d'abord la création de la pioche complète, avec chaque personnage associé à un nombre allant de 1 à 9 (le nombre des différents personnages). Puis la deuxième étape était de piocher une des 72 tuiles au hasard, en l'effaçant de la mémoire de la pioche par la suite pour rendre impossible le fait de piocher plusieurs fois le même couple.

Une partie originale de ce projet fut la décision d'afficher et d'enregistrer des grades obtenus en fin de partie en fonction du nombre de pièces obtenues. Cette fonction était prévue dans le cahier des charges mais pas dans le sujet. Ces grades existent en fait dans le mode Solo du jeu, et nous avons décidé de les inclure dans notre code en changeant quelques noms et le nombre de pièces à obtenir pour les décrocher.



Ecran des statistiques

Problèmes rencontrés et solutions apportées

Un des problèmes rencontrés fut la création d'une nouvelle page à chaque fois qu'on cliquait sur un bouton dans le menu. Pour résoudre ce problème, nous avons décidé d'opter pour une solution simple ; à chaque fois qu'un bouton est cliqué, nous affichons une image de fond par-dessus, puis nous affichons les nouveaux boutons et les textes par-dessus ce fond. Cela donne une illusion d'une nouvelle page ouverte alors qu'en réalité, ce n'est que superposer tous les éléments de la page initiale.

Un autre problème fut la sauvegarde des comptes. Tout d'abord, nous avons dû faire le choix entre une base de données ou un fichier. Nous avons finalement décidé d'opter pour le fichier en raison de l'unicité de la table et du peu d'informations qu'elle contient. La difficulté rencontrée était ici la sauvegarde des comptes. C'est-à-dire que l'inscription ou la connexion des comptes selon qu'ils existaient ou non dans le fichier fut simple à coder pour nous. Le problème résidait donc dans la sauvegarde de ce fichier. Si on fermait la fenêtre et relançait le programme, les comptes créés dans l'itération précédente n'existaient plus. La solution que nous avons trouvée pour ce problème est la suivante : nous avons créé les classes Joueur et GestionJoueurs pour sauvegarder toutes les informations dans un fichier.

La règle du jeu de société spécifie que lorsqu'on pose une tuile, elle tombe automatiquement en bas de la colonne (juste au-dessus de la dernière tuile présente dans la colonne s'il y en a une). La réponse à cela a été la création de la classe Case qui permet de connaître l'état de chaque case présente sur la grille. Ainsi lorsqu'on clique sur une colonne, la classe Grille va nous permettre de calculer où est-ce qu'on a cliqué, puis de vérifier si la case d'en dessous est occupée ou non (grâce à la classe Case). Si ce n'est pas le cas, on vérifie celle d'en dessous etc., jusqu'à trouver une case libre pour poser la tuile que l'on a piochée.

Un personnage a également été plus difficile à gérer que les autres, le singe. Il est écrit dans la règle qu'il permet de flipper (donc de retourner) une seule des tuiles présentes autour de lui. Pour réaliser cette action nous avons choisi de créer une variable du nom de "mode singe". Cette variable passe à "True" uniquement s'il existe au moins une case autour du singe que l'on peut retourner. S'il n'existe aucune case à retourner, on gagne simplement une pièce (qu'on gagnera dans tous les cas) et on ne passe pas en "mode singe".

Les écarts par rapport au cahier des charges

Dans le cahier des charges, il était initialement prévu de créer un joueur ordinateur, ainsi que potentiellement ajouter des niveaux de difficulté. Cette partie n'a pas pu être faite à cause principalement d'un manque de temps, la difficulté du projet nous ayant pris la grande majorité du temps prévu pour sa résolution, mais aussi à cause de la difficulté de créer un joueur ordinateur capable de faire des choix logiques dans l'optique de gagner, et non totalement au hasard.

Nous avons aussi initialement prévu d'ajouter un bandeau lors de l'obtention de pièces, avec écrit par exemple "Vous avez gagné [X] pièces !", ainsi qu'une bourse cliquable pour consulter son nombre de pièces. A la place, nous avons préféré opter pour une variable Gain qui se trouve au-dessus du plateau du joueur et qui garde en temps réel le compte des pièces de ce joueur. Cette solution nous permet de ne pas surcharger l'écran de jeu ainsi que de simplifier les informations pour les joueurs.

Une autre fonctionnalité qui était initialement prévue était de proposer aux joueurs de choisir sur quel plateau ils voulaient jouer (plusieurs plateaux étant disponibles dans le jeu). Nous nous sommes finalement concentrées sur un seul plateau pour être sûres d'avoir un plateau de jeu fonctionnel sur lequel on pouvait faire une partie complète.



Encore une autre fonctionnalité était la création d'un bouton "Aide" qui aurait été présent sur la table de jeu. En cliquant sur ce bouton, nous voulions afficher sur l'écran une aide de jeu, qui consiste en l'explication de ce que font les personnages. Cette aide aurait pu être utile non seulement pour savoir ce que fait le personnage actuellement pioché, mais aussi pour aider le joueur à décider s'il voulait flipper la tuile ou non.

Un écart avec le cahier des charges très important est la liste des classes, fonctions et variables utilisées. Beaucoup d'entre elles ne figurent pas dans ce que nous avons prévu. Cela s'explique simplement par la difficulté de prévoir des outils si précis sans avoir le recul d'avoir déjà travaillé sur un projet de cette ampleur.

Bilan : Ce que le projet nous a apporté

Les +

Ce projet nous a tout d'abord apporté une connaissance de nouvelles utilisations de Python, notamment grâce à la bibliothèque PyGame, permettant de coder des jeux vidéos, et ses différents modules (`pygame_gui`, `pygame.display`, `pygame.image`, `pygame.transform`, etc...), qui servent à faire fonctionner l'interface graphique et la modifier à notre guise pour afficher ce que l'on veut à l'utilisateur.

Il nous a aussi appris à rédiger un cahier des charges, nous avons ainsi pu comprendre la difficulté qui est non seulement de créer ce fichier sans avoir de recul sur le code que l'on ne fera que plus tard, mais aussi de s'y tenir au fil du projet. Nous avons pu évoquer dans une partie de ce rapport ce qui a différé entre le cahier des charges rédigé au début du projet et notre code final.

Nous avons aussi appris à gérer beaucoup d'informations en même temps : la gestion de l'affichage de la table de jeu, de la grille, de la tuile que l'on pioche, des éventuels effets de cette tuile, des différents boutons, etc...

Le fait d'apprendre à segmenter notre travail pour arriver au résultat final nous a facilité la tâche et nous servira plus tard.

Les -

Malheureusement, nous n'avons pas pu réaliser autant de fonctionnalités que prévues et voulues, le temps imparti semblant trop court par rapport à la difficulté du sujet.

Quelque chose de très difficile dans ce projet fut la réitération d'un problème en particulier : le fait de coder une nouvelle fonctionnalité peut parfois avoir pour conséquence d'empêcher quelque chose qui a été codé précédemment de fonctionner. Ce problème récurrent est très décourageant à chaque fois qu'il est rencontré.

Conclusion

Discussion des résultats

Nous avons un programme complètement fonctionnel qui remplit la quasi-totalité des demandes du sujet (la seule différence étant qu'on ne peut pas jouer contre un joueur ordinateur mais uniquement contre un joueur humain). Le programme fonctionne comme voulu, la gestion des comptes fonctionne et nous pouvons jouer au jeu jusqu'à la fin de partie. Nous pouvons aussi décider de relancer une partie avec les mêmes joueurs déjà connectés en cliquant sur le message de victoire qui apparaît à la fin d'une partie.

En conclusion, ce projet a été une expérience de développement qui nous a permis de maîtriser de nouveaux outils, notamment la bibliothèque PyGame. Malgré les obstacles rencontrés, comme la gestion complexe des tuiles et des comptes joueurs, ainsi que les ajustements nécessaires par rapport au cahier des charges initial, nous avons réussi à créer un jeu fonctionnel. Ce projet nous a non seulement permis de développer nos compétences en programmation et en gestion de projet, mais il a également démontré l'importance de la planification et de l'adaptabilité dans l'univers du développement informatique. Les connaissances acquises et les leçons tirées de cette expérience seront utiles pour nos futurs projets, qu'ils soient professionnels ou personnels.

Prolongements possibles

N'ayant pas pu faire le joueur ordinateur, un prolongement possible serait de pouvoir le créer. Notre seule idée pour faire fonctionner ce joueur ordinateur était de faire fonctionner ses choix entièrement sur le principe du hasard. Par exemple, en choisissant de retourner la tuile ou non à l'aide d'un chiffre tiré au hasard entre 1 (pour ne rien faire) et 2 (pour retourner la tuile), ou encore le choix de la case où la tuile serait posée serait aussi fait au hasard (un chiffre entre 1 et 5, 5 étant le nombre de colonnes). Il faudrait alors vérifier quand est-ce qu'une colonne est complète pour retirer ce chiffre des choix possibles, afin de toujours avoir une case libre tirée au sort.

Un autre prolongement possible serait, comme suggéré dans le cahier des charges rédigé au début de ce projet, de pouvoir choisir le plateau sur lequel nous voulons jouer et donc de coder les cases et les bonus qui sont différents de ceux du premier plateau.

La fonctionnalité de pouvoir jouer à plus que deux (et donc d'afficher plus de plateaux ainsi que plus de comptes dans l'onglet statistiques) est aussi un des prolongements possibles.

Si l'on joue à plus que deux, il faudra en plus coder le cas de victoire qui correspond à une égalité entre deux joueurs, sachant qu'aucun des deux ne possède la carte. Dans ce cas, il est explicité dans les règles du jeu que ces deux joueurs se partagent la victoire.

Nous pouvons également ajouter la possibilité de supprimer un joueur des comptes.

Améliorations

Pour les améliorations, nous avons tenté d'inclure un bouton "Retour" dans les pages où l'on pouvait entrer du texte pour se connecter et s'inscrire. Mais nous n'avons pas réussi à faire fonctionner et le rectangle de rentrée de texte et le bouton "Retour" à la fois. Nous rencontrons toujours le problème que l'un des deux ne fonctionnait pas quand l'autre fonctionnait. Une amélioration possible serait donc de réussir à faire fonctionner ces deux outils en même temps pour permettre à l'utilisateur de revenir en arrière s'il clique sur "Se connecter" ou "S'inscrire". En l'état actuel des choses, nous ne pouvons pas revenir en arrière, la seule solution étant de fermer la page et de relancer le programme.

Une autre amélioration possible serait d'animer les pièces gagnées au cours de la partie ou d'afficher le gain obtenu (exemple : afficher +2p quand on pose une tuile qui nous rapporte deux pièces). Nous pourrions aussi afficher une bourse cliquable avec les pièces obtenues visibles à l'intérieur.

Enfin, nous pourrions optimiser notre code en mettant chacune des classes dans un fichier différent (exemple : les classes Grille et Case se trouvent dans la boucle Jeu alors que les classes Joueur, Gestion Joueur et Pioche se trouvent dans des fichiers séparés).